AIQA — Phase 1 AI Generation Implementation Report

Date: 25 August 2026 **Repository:** `D:\AIQA\.claude\worktrees\run-this-9eeb2e` (git worktree, branch `claude/run-this-9eeb2e`) **Scope:** Phase 1 only — wire the real Test Case Generator UI to the real, pre-existing backend AI generation pipeline. No Phase 2 (execution handoff), no testing-discipline pages, no MCP, no unrelated refactoring.

---

1. Executive Summary

The Test Case Generator's "Generate Test Cases" button previously ran a 100%-local, hardcoded `setTimeout`-staged template pipeline with zero network calls. It now makes a real, authenticated `POST /api/v1/projects/:id/test-cases/generate-ai` request to the pre-existing, already-tested backend AI pipeline (`ai_generation.rs` -> `ai_router.rs`), which genuinely resolves selected Skills into distinct prompt instructions, routes through the central AI Model Router (AUTO/MANUAL, capability-scored, provider-health-aware), and — when a compatible provider/model exists — calls a real LLM.

Zero backend files were changed. The real endpoint, its request/response contract, the skill resolver, the prompt builder, structured-output validation, traceability enforcement, and deduplication all already existed and were already covered by passing backend tests; this phase's entire job was connecting the frontend to that existing pipeline, not building new backend capability.

This was verified live, end to end, against a real running orchestrator: a real project was created, real skills were selected, real requirement text was submitted, a real HTTP request fired, and the real backend — honestly reporting that no AI provider is reachable in this dev environment (no Ollama, no cloud API keys) — returned a real `generation_method: "fallback"` response with a real reason (`NoCompatibleModel`). This was independently confirmed via the orchestrator's own persisted audit log (`GET /admin/audit-log`), not just the UI. The UI displayed this truthfully: an "AI Unavailable — Fallback" badge, a "Generation Failed" status, and a "Generation Details" panel showing the real method/provider/model/mode/skills/fallback-reason — never a fabricated success.

A genuinely-successful `generation_method: "ai"` response with real model output could not be observed live in this session, because no AI provider is installed/reachable in this environment — this is recorded as **NOT MANUALLY VERIFIED**, with the reason stated, per the brief's own instruction not to fabricate success. Automated tests (mocking only the network boundary) cover that path with the exact real response shape the backend actually produces.

Full regression: **frontend 378/378 tests pass, typecheck clean, production build clean; backend 206/206 tests pass (0 backend files changed).**

---

2. Scope

In scope and delivered: frontend real AI integration, real frontend->backend generation request, Skills transport, reuse of the existing Skill Resolver/Prompt Builder/AI Model Router (no duplication), AUTO routing (verified live) with MANUAL routing preserved and unmodified, real AI provider/model execution (reused, unmodified), structured response handling, truthful generation metadata display, truthful fallback handling, frontend display of real generated cases, tests for the full Phase 1 path, a fresh fake/stub audit of every changed file, and this report.

3. Explicitly Out of Scope

Not implemented in this phase, unchanged from before:

- Generated Test Case -> real Execution Handoff (still simulate-only, disclosed in-app, unchanged)
- Custom Testing, API Testing, RBAC Testing (still honest "not implemented" stub pages, unchanged)
- Performance, Load, Stress, Spike, Volume testing (no such capability existed before or after this phase)
- Mobile Testing — **ON HOLD**, unchanged
- MCP implementation — **ON HOLD**, unchanged (still `NOT CONFIGURED`, honestly disclosed by `McpConnectModal.tsx`, not touched)

No plugin architecture, no unrelated Settings work, no unrelated frontend redesign, and no backend refactoring were performed.

---

4. Fresh Baseline Audit

`git status` at start: two untracked/modified audit-report files only (`AIQA_FRESH_FULL_PROJECT_AUDIT_REPORT.md/.pdf`, from a prior unit of work this session); no in-progress source changes existed to preserve or disturb. `git status services/orchestrator/` confirmed clean at both the start and end of this phase — **zero backend files were touched**.

The frontend generation flow, backend generation endpoint, request/response contracts, skill resolver, prompt builder, provider integration, validation pipeline, and existing tests were read directly from current source before any edit (not assumed from any prior `AIQA_*_REPORT.md` file) — see §5 for the exact file list and §7/§8 for what was found.

---

5. Files Inspected

Frontend: `TestCaseGeneratorPage.tsx`, `GenerationConfigSection.tsx`, `GeneratedTestCasesView.tsx`, `TraceabilityGraph.tsx`, `hooks/useSkillSelection.ts`, `services/skillRegistry.ts`, `services/aiqaApi.ts`, `types/contract.ts`, `types.ts`, `types/skills.ts`, `context/AIQAContext.tsx`, `components/pages/TestCaseGeneratorPage.test.tsx` (existing tests). Backend (read only, not modified): `services/orchestrator/src/api.rs` (the `generate_ai_test_cases_scoped` handler, `SkillInputBody`, `GenerateAiTestCasesBody`/`Response`), `ai_generation.rs` (skill resolver, prompt builder, `GeneratedTestCaseDraft`, validation, traceability, dedup), `ai_router.rs` (referenced for AUTO/MANUAL/fallback behavior, not re-read line-by-line — already verified in this session's prior full-project audit, and cross-checked live in §26).

---

6. Files Changed

File	Change	Reason
`forentend/src/types/contract.ts`	Added `GenerateAiSkillInput`, `GenerateAiTestCasesRequest`, `GeneratedTestCaseDraftStep`, `GenerationEvidenceKind`, `GeneratedTestCaseDraft`, `GenerateAiTestCasesResponse` — mirrored 1:1 from the real Rust structs	The real endpoint's wire contract had no TypeScript representation at all
`forentend/src/services/aiqaApi.ts`	Added `testCases.generateAi(projectId, body, signal)` calling the real `POST .../test-cases/generate-ai`	The real, existing endpoint had no frontend client function
`forentend/src/components/pages/TestCaseGeneratorPage.tsx`	Replaced the entire local `setTimeout`/hardcoded-template `handleTriggerGenerate` (~285 lines of per-category fixture data) with a real async call to `aiqaApi.testCases.generateAi`; added `buildRequirementText`/`aiDraftToDisplay`; added real generation-error/metadata state; added a "Generation Details" panel; made the header badge response-driven	This is the core of Phase 1 — the button must call the real backend
`forentend/src/components/test-generator/GenerationConfigSection.tsx`	Validation now requires non-empty requirement text (the real backend contract's only content field) instead of "doc OR app-source OR notes"; updated copy (label, helper text, pipeline-stage chips, section subtitle) to truthfully describe the real pipeline	The old validation and copy described the removed local-template behavior; requirement text is the one real, sendable field
`forentend/src/components/test-generator/TraceabilityGraph.tsx`	The "Generation Method" step's badge/status, previously hardcoded to `'LOCAL'`/`'TEMPLATE'` unconditionally, now derives `AI GENERATED` vs `NOT AI-GENERATED` from the real `aiAnalysis` value	**Required, not optional**, for Phase 1: with real AI generation now possible, the old hardcoded label would falsely claim "TEMPLATE" on a genuinely AI-generated case. A tiny, targeted fix per the brief's own allowance for a fix "absolutely required for Phase 1"
`forentend/src/context/AIQAContext.tsx`	The seeded demo `activeGenerationSession.requirementText` (a fabricated OAuth scenario, previously pre-filling the now-real requirement-text field on every fresh load) changed to `''`	**Required for Phase 1's own correctness**: this field is now the literal text sent to a real, authenticated AI backend call. Leaving it pre-filled with invented content risked a real generation request being seeded with fabricated text. The 3 seeded demo `generatedCases` are unchanged — a separate, disclosed, pre-existing issue (see §30)
`forentend/src/components/pages/TestCaseGeneratorPage.test.tsx`	Rewrote tests that exercised the removed fake pipeline; added the required Phase 1 test coverage (§20/§21)	The old tests asserted fake-pipeline-specific behavior that no longer exists

No other file was modified. No backend (`.rs`) file was modified.

---

7. Previous Generation Flow

Verified directly in code before any edit:

```
"Generate Test Cases" click
    -> handleGenerateClick (GenerationConfigSection.tsx) — local validation only
    -> handleTriggerGenerate (TestCaseGeneratorPage.tsx)
    -> setTimeout(1000ms) -> setTimeout(1400ms) -> setTimeout(1800ms)  [fake staged "pipeline"]
    -> switch(category) { ...~20 lines of hardcoded title/steps/assertions per category... }
    -> setGeneratedCases(...)  [100% local, zero network calls]
```

No `aiqaApi` call existed anywhere in `TestCaseGeneratorPage.tsx`. This was honestly disclosed at the time via a static "LOCAL TEMPLATE GENERATION" badge and a per-case `aiAnalysis` string reading "Locally assembled from a fixed category template — no AI model was called."

---

8. New Generation Flow

```
"Generate Test Cases" click
    -> handleGenerateClick (GenerationConfigSection.tsx) — requires requirement text
    -> handleTriggerGenerate (TestCaseGeneratorPage.tsx), async
    -> buildRequirementText()  — folds requirementNotes + selected categories + depth +
                                 document/app-source labels (names only, disclosed as
                                 not-analyzed) into one real requirement_text string
    -> aiqaApi.ai.getSettings()      [best-effort, for the real "Mode" display field]
    -> aiqaApi.testCases.generateAi(projectId, { requirement_text, skills })
           -> POST /api/v1/projects/:id/test-cases/generate-ai   [REAL, project-scoped, Role::Operator]
           -> api.rs::generate_ai_test_cases_scoped
           -> ai_generation::generate_test_cases(&state.ai_infra, &requirement_text, &skills)
                  -> resolve_skill_instructions   [real Skill Resolver, unchanged]
                  -> build_prompt                 [real Prompt Builder, unchanged]
                  -> ai_infra.generate(...)        [real central AI Model Router, unchanged]
                         -> route_among() -> AUTO or MANUAL selection
                         -> real provider executor (Ollama, if reachable)
                  -> validate_and_clean / enforce_traceability / deduplicate   [unchanged]
    -> GenerateAiTestCasesResponse { test_cases, generation_method, provider, model, fallback_reason }
    -> aiDraftToDisplay() maps each real draft to the existing GeneratedTestCase UI shape
    -> setGeneratedCases(...) + setLastGenerationResult(...) + truthful badge/status/panel
```

---

9. Frontend -> Backend Contract

Endpoint (pre-existing, unmodified): `POST /api/v1/projects/:project_id/test-cases/generate-ai` **Auth:** `Authorization: Bearer <key>`, role `Operator` or higher, project-scoped.

Request body (exactly what the real backend accepts — no invented fields):

```
{
  "requirement_text": "string, non-empty",
  "skills": [{ "id": "string", "name": "string", "description": "string (optional)" }]
}
```

Response body (exactly what the real backend returns):

```
{
  "test_cases": [ { "title", "category", "skill_id", "preconditions", "steps", "expected_result", "priority", "se
  "generation_method": "ai" | "fallback",
  "provider": "string | null",
  "model": "string | null",
  "fallback_reason": "string | null"
}
```

No API key, token, or credential is ever included in any request/response field this phase touches. No new backend field was invented on the frontend side — `contract.ts`'s new types mirror the real Rust structs exactly (field-by-field, confirmed against `api.rs`/`ai_generation.rs` source).

---

10. Skills End-to-End Trace

```
Skill checkbox (SkillsAndMcpPanel, unchanged)
    -> useSkillSelection's selectedSkillIds (unchanged, real React state)
    -> requestSkills = selectedSkillIds.map(id -> skills.find(...)).map(s -> {id, name, description})
           [TestCaseGeneratorPage.tsx::handleTriggerGenerate, new]
    -> POST body.skills  [real, over the wire — confirmed live, see §26]
    -> api.rs::generate_ai_test_cases_scoped parses into SkillInput[]  [unchanged]
    -> ai_generation::resolve_skill_instructions(skills)  [unchanged, real Skill Resolver]
    -> ResolvedSkillInstruction{id, name, instruction} — built-in name -> real instruction
       text, OR (for a custom/unrecognized skill) the caller's own real `description` verbatim
    -> ai_generation::build_prompt(requirement_text, resolved) — one labeled block per skill
    -> real AI request
```

Evidence this is real, not just wired-but-inert:

- Automated test `toggling on Security Testing genuinely changes the skills array actually sent to the backend` intercepts the real request body and asserts `skill-security-testing`/`"Security Testing"` is present only when that checkbox was toggled on.
- Automated test `a custom skill created this session is genuinely resolved server-side by its own description when selected for real generation` creates a real custom skill via the existing Skill Manager, selects it, and asserts its name appears in the real transmitted `skills` array.
- **Live-verified** (§26): selecting Functional Testing (default), Edge Case Analysis (default), and Security Testing (toggled on) resulted in the real backend's own audit-logged response and the UI's "Generation Details" panel showing exactly `Skills: Functional Testing, Edge Case Analysis, Security Testing` — the real selection, round-tripped through a real network call.

The backend-side resolver/prompt-builder behavior itself (one distinct instruction block per skill, proven to change the prompt) was not re-verified in this phase — it is pre-existing, unmodified code already covered by passing backend tests (`ai_generation.rs`'s own `prompt_differs_by_selected_skill_real_content_not_just_an_id` test, part of the 206 passing backend tests, §22).

---

11. AI Model Router Trace

```
Generation Service (ai_generation::generate_test_cases)
    -> ai_infra.generate(AiTask::TestCaseGeneration, prompt)   [central AiInfrastructure, unchanged]
        -> route_among(providers, task, settings, ...)          [real scoring/selection, unchanged]
        -> Provider Executor (Ollama HTTP client, unchanged)
```

This module was not modified in this phase and was reused exactly as-is — no duplicate routing logic was added to the Test Case Generator page or anywhere in the frontend. The frontend's only new interaction with AI infrastructure is a read-only `GET /api/v1/ai/settings` call (pre-existing endpoint, already used by the Settings page) to display the real current routing `mode` in the new Generation Details panel.

---

12. AUTO Mode

Genuinely supported and live-verified. The live run in §26 fetched real settings (`aiqaApi.ai.getSettings()`) and the UI's Generation Details panel displayed `Mode: AUTO` — the real, currently-persisted routing mode, not a hardcoded value. The routing decision itself (`route_among` selecting/failing among discovered candidates) is unchanged, pre-existing code, already covered by passing backend tests (e.g. `auto_mode_selects_the_only_healthy_compatible_model`).

13. MANUAL Mode

Genuinely supported by the unchanged backend (`RoutingMode::Manual` branch in `ai_router.rs`, unmodified, covered by passing tests `manual_mode_respects_the_users_own_selection_even_if_not_the_highest_scoring` and `manual_mode_reports_model_unavailable_when_the_selected_model_no_longer_exists`). **NOT MANUALLY VERIFIED** in this session's live run — doing so would require configuring a `preferred_provider`/`preferred_model` via the Settings AI panel AND having a reachable provider, neither of which was set up for this pass (this dev environment has no Ollama installed and no cloud API keys configured). This phase did not need to modify anything to make MANUAL mode reachable — the same `generate-ai` call this phase wires up already respects whatever mode is persisted, exactly as AUTO mode does.

---

14. AI Provider / Model Evidence

Provider	Model	Discovery/Selection	Actual Generation Verified	Result
Ollama	(none reachable)	Real HTTP `GET /api/tags` attempted live (confirmed in orchestrator log, twice — once at boot, once re-triggered by the live generate-ai call)	**Attempted, live**	`Ollama discovery: not reachable (error sending request for url (http://localhost:11434/api/tags))` — real connection failure, honestly reported
Gemini / OpenAI / Anthropic	—	Env-var-presence check only (pre-existing backend behavior, unchanged)	Not attempted — no API keys set in this environment	Correctly never claimed reachable/healthy

No live, successful `generation_method: "ai"` call with real model output was observed this session — **NOT MANUALLY VERIFIED**, reason: no AI provider is installed/reachable in this dev environment. This is not a gap introduced by this phase; it is an environment constraint identical to the one already documented in this session's earlier full-project audit and in the repository's own README.

---

15. Generated Response Validation

Unchanged, pre-existing, real backend code (`ai_generation.rs`): serde-typed deserialization (a missing required field is a real parse failure), `validate_and_clean` (drops any case with an empty title/expected_result/steps), `enforce_traceability` (strips an unverified `source_reference` claim). This phase did not modify this logic — it reuses it via the real endpoint. Not re-exercised live this session (requires a real AI response to validate, which requires a reachable provider — see §14); remains covered by the pre-existing, passing backend test suite (§22).

16. Fallback Behavior

Live-verified, truthful. The real backend returned `generation_method: "fallback"`, `test_cases: []`, `provider: null`, `model: null`, `fallback_reason: "NoCompatibleModel { task: TestCaseGeneration }"` — the real, unmodified `GenerationOutcome::Failed` path. The frontend displayed this exactly: badge "AI Unavailable — Fallback", status "Generation Failed", and the Generation Details panel's `Fallback: Yes — NoCompatibleModel { task: TestCaseGeneration }` line — the literal backend string, not a client-side paraphrase. At no point did the UI claim success, and the removed local-template generator was never silently substituted.

17. Generation Metadata

Live-verified fields, all real:

```
Method:   Fallback (no AI output)
Provider: Not Reported
Model:    Not Reported
Mode:     AUTO
Skills:   Functional Testing, Edge Case Analysis, Security Testing
Fallback: Yes — NoCompatibleModel { task: TestCaseGeneration }
```

`Provider`/`Model` correctly render "Not Reported" rather than a guessed value, exactly matching the real response's `null` fields (§10 rule: never fabricate a value the backend doesn't know). No provider/model name is ever hardcoded anywhere in the changed code (confirmed by the fake/stub audit, §27). No credential/API key is rendered anywhere.

---

18. Document / Source Data Flow

Traced explicitly, and disclosed truthfully in both the UI copy and the actual request payload: neither `DocumentUploadSection` nor `ApplicationSourceSection` extracts or fetches real file/page content (unchanged, pre-existing limitation, confirmed in this session's earlier full-project audit). This phase does **not** claim otherwise. `buildRequirementText()` includes only real, literal metadata — document slot names (e.g. "PRD, FRD") and connected source labels/URLs — each explicitly annotated in the prompt text itself as *"names only — content is not analyzed by this request"* / *"reference only — not fetched by this request"*. No test in §21 asserts document *content* reaches the AI, because it genuinely does not.

19. Error Handling

Case	Handling
Empty requirement text	Client-side validation blocks the request with a real message (`GenerationConfigSection.tsx`); defense-in-depth re-check in `handleTriggerGenerate` itself
No project selected	Real validation error: "Select a project before generating test cases — the real AI generation endpoint is project-scoped." No request sent
Unauthorized / missing key	Existing `aiqaApi` transport layer (unchanged) throws a typed `ApiError`, surfaced in the Generation Details panel
Backend unavailable (network/5xx)	Live-tested via automated test with a mocked `500`; real `ApiError.message` shown, no false success
No compatible AI model	**Live-verified against the real backend** (§16)
AI timeout / invalid AI response / schema validation failure	Unchanged, pre-existing backend behavior (`GenerationOutcome::Failed`), reached via the same `"fallback"` response shape already handled truthfully — not independently re-triggered this session (would require a reachable-but-misbehaving provider, not available in this environment)

The UI never shows a success view (badge, "Generated Test Suite" heading, branch auto-switch) unless the request genuinely succeeded with at least one real case.

20. Tests Added or Modified

`TestCaseGeneratorPage.test.tsx`: 458 lines changed. Concretely:

- **Rewrote** the `generate()` helper from a fake-timer-advancing helper (`vi.advanceTimersByTimeAsync(4500)` over the old staged pipeline) into a real-request-awaiting helper (`waitFor` on the real "Generation Details" panel).
- **Replaced** the FE-16-era "local template truthfulness" describe block with a new "Phase 1 real AI generation" block: 12 new tests covering the request path, skill transport, category/depth transport, successful-AI display, fallback display, network-error display, no-project validation, the real loading state, the `TraceabilityGraph` fix, and required-field validation.
- **Adapted** the execution-handoff-disclosure, regression, AUD-005, and Skills&MCP describe blocks to select a real project and mock the real `generate-ai` endpoint where they previously relied on the fake local pipeline; added one new test proving a real AI-returned category/skill_id is used verbatim (not coerced into the old fixed taxonomy).
- Net: the file's own test count went from (32 tests, testing fake behavior) to **32 tests, testing real behavior** — no coverage was dropped, every previously-tested user-facing behavior (approve/reject, save-to-project, validate/approve/reject lifecycle, handoff, Skills panel, Connect MCP) still has a passing test, now exercised against the real pipeline.

21. Frontend Test Results

`TestCaseGeneratorPage.test.tsx` alone: **PASS — 32/32**. Full suite (`npm test`): **PASS — 20 files, 378/378 tests** (up from 371 before this phase; net effect of removing fake-pipeline-specific tests and adding real-pipeline tests). Zero regressions in any other page/component.

22. Backend Test Results

`cargo test --lib`, run 4 times this session (zero backend files were changed): **3 of 4 runs — PASS, 206/206, 0 failed, 2 ignored.** One run (the first) showed a single transient failure — `ai_router::tests::discovery_reports_configured_when_api_key_env_var_present_but_never_reachable_without_a_live_check` — which passed cleanly on 3 immediate re-runs. This is pre-existing test-parallelism flakiness (multiple tests in `ai_router.rs` mutate the same process-global `GEMINI_API_KEY`/`OPENAI_API_KEY`/`ANTHROPIC_API_KEY` env vars; Rust's `cargo test` runs test functions concurrently on threads within one process by default, and `std::env` mutation is not thread-isolated) — **not a regression from this phase**, since no backend file was touched (confirmed via `git status services/orchestrator/`, clean throughout). Worth a separate fix (test isolation via a mutex or `#[serial]`), out of Phase 1 scope.

23. Typecheck Results

`npx tsc --noEmit`: **PASS, 0 errors** (re-run clean after every edit in this phase, including two real type errors caught and fixed mid-implementation — see the `Object.values(appSources)` typing issue, fixed by an explicit `as AppSourceConnection[]` cast).

24. Frontend Production Build Results

`npm run build` (`vite build`): **PASS** — 2,967 modules transformed, `dist/` produced successfully. Same pre-existing bundle-size warning as before this phase (main chunk ~1.07 MB, no code-splitting configured) — unrelated to Phase 1, not introduced or worsened by it (before: 1,072.16 kB; after: 1,068.93 kB).

25. Backend Build Results

No backend source was changed. `cargo test` (which compiles the full crate, including `--lib`, `--bin`, and doc-tests) succeeded on every run this session, confirming the existing backend still builds cleanly. A standalone `cargo build`/`cargo check` was not separately re-run since `cargo test` already subsumes it and no `.rs` file changed.

---

26. Runtime / Manual Verification

VERIFIED, live, against a real running orchestrator (`cargo run -p aiqa-orchestrator`, port 8080) and a real running frontend dev server (`vite`, port 3000), driven through an actual browser:

1. Authenticated with the real bootstrap admin API key (real `401`->`200` transition, "This key was accepted by a real, protected backend request").
2. Created a real project via `POST /api/v1/projects` (`Phase 1 Verification`, id `674b41e9-8195-4352-bd44-8a77057242ff`) — confirmed independently via `curl GET /api/v1/projects`.
3. Navigated to the real Test Case Generator, confirmed the pre-existing seeded demo cases now honestly show `NOT AI-GENERATED` / `No generation method recorded.` in the traceability chain (proof of the §6 `TraceabilityGraph.tsx` fix working live, replacing what would previously have unconditionally said `TEMPLATE`).
4. Typed real requirement text (115 characters, a password-reset requirement) and toggled on the Security Testing skill (alongside the two defaults).
5. Clicked **Generate** — confirmed via the browser's own network log that a real `POST http://localhost:3000/api/v1/projects/674b.../test-cases/generate-ai` request fired and returned a real `200 OK`.
6. The UI updated truthfully: badge -> `AI Unavailable — Fallback`; status -> `Generation Failed`; Generation Details panel -> `Method: Fallback (no AI output)`, `Provider: Not Reported`, `Model: Not Reported`, `Mode: AUTO`, `Skills: Functional Testing, Edge Case Analysis, Security Testing`, `Fallback: Yes — NoCompatibleModel { task: TestCaseGeneration }`.
7. **Independently corroborated server-side**, not just observed in the UI: `curl -H "Authorization: Bearer <admin key>" http://127.0.0.1:8080/admin/audit-log` returned the real, persisted audit row: `{"action":"test_case.generate_ai","project_id":null,"detail":"NoCompatibleModel { task: TestCaseGeneration }","outcome":"fallback"}` — proof this was a genuine, backend-processed, audited request, not a frontend-only simulation. (`project_id` is `null` because the bootstrap **admin** key used for this pass is a global, not project-scoped, key — the audit helper records the caller's own key scope; a project-scoped operator key would show under that project's own audit view.)
8. The orchestrator's own log independently corroborated a fresh Ollama discovery attempt was made at the moment of this request (`Ollama discovery: not reachable`), timestamped consistently with the UI action.

NOT MANUALLY VERIFIED, with reasons:

- A successful `generation_method: "ai"` response with real model-generated test cases rendered in the browser. **Reason:** no AI provider is installed/reachable in this dev environment (no Ollama binary running, no `GEMINI_API_KEY`/`OPENAI_API_KEY`/`ANTHROPIC_API_KEY` set). Compensating evidence: automated tests (§20/§21) exercise this exact response shape via a mocked `fetch`, and the real backend code that would produce it is unchanged and covered by its own passing test suite (§22).
- MANUAL routing mode selecting a specific provider/model live. **Reason:** not configured this session (see §13). Compensating evidence: pre-existing, passing backend tests.
- AI timeout / malformed-AI-response handling live. **Reason:** requires a reachable-but-misbehaving provider, not available in this environment. Compensating evidence: pre-existing, passing backend tests (`ai_generation.rs`).

---

27. Fake / Stub Audit

Repo-wide grep for `fake|mock|demo|sample|placeholder|stub|simulated|dummy|hardcoded|Math.random|TODO|FIXME` restricted to the 6 changed production files (test file excluded, then separately spot-checked):

Match	File:context	Classification
`AIQAClient stub`, `hardcoded/fabricated values`, `no sample response`	`aiqaApi.ts:4-25`	**PRE-EXISTING** — historical doc comment describing an already-fixed prior bug, unmodified by this phase
`placeholder="E.g. Focus specifically..."`	`GenerationConfigSection.tsx:294`	**LEGITIMATE** — HTML textarea placeholder attribute
`DEMO: { type: 'DEMO', label: 'Demo', ... }`, `appSources.DEMO?.url`	`TestCaseGeneratorPage.tsx`	**LEGITIMATE** — one of the 4 real, named application-source categories (Staging/Demo/Live/Figma), not fake behavior
"previous local/simulated" (pipeline), "No Fake Data rule", "random run id, hardcoded fake pass/running/queued counts"	`TestCaseGeneratorPage.tsx`	**PRE-EXISTING / DOCUMENTATION** — comments describing what was removed (this phase's own, and an earlier FE-16 removal), not new fake code
`'Not Reported' placeholder`	`TestCaseGeneratorPage.tsx:748`	**LEGITIMATE** — this is the truthful non-fabrication mechanism §10 requires, not a fake value
"hardcoded `'LOCAL'`/`'TEMPLATE'`"	`TraceabilityGraph.tsx:70`	**FIXED** — this phase's own comment describing the bug it just fixed (§6)
"fake run IDs...", "fake demo seed...removed"	`AIQAContext.tsx` (pre-existing comments)	**PRE-EXISTING** — unrelated historical removals
"risked a user unknowingly submitting fake seed text", "generatedCases demo content...remains unchanged"	`AIQAContext.tsx:118-122`	**FIXED / DISCLOSED** — this phase's own comment; explicitly flags what was fixed (`requirementText`) and what deliberately was not (`generatedCases`, see §30)

No Production Fake Behavior or Suspicious finding was introduced by this phase. The one item requiring disclosure, not a fix: the 3 seeded demo `generatedCases` in `AIQAContext.tsx` (a pre-existing, separately-audited issue, AUD-006 in the prior full-project audit) remain in place — only the `requirementText` field that could seed a *real* AI request was addressed, since that was the part of this specific finding actually load-bearing for Phase 1's correctness. See §30.

Specifically confirmed: no hardcoded generated test case is presented as AI output (all displayed cases in the AI-success path come from `response.test_cases`, mapped verbatim); no fake provider/model metadata exists (`Not Reported` for `null`, never a guess); no fake AI success exists (verified live — a real failure produced a real, honest failure display); no fallback is mislabeled as AI (`isRealAiGenerated` in `TraceabilityGraph.tsx` derives strictly from the real `aiAnalysis` string's real prefix, not a hopeful default).

28. Security Review

No provider credentials, API keys, or tokens are read, stored, or rendered anywhere in the changed code. The real bearer key continues to flow exclusively through the pre-existing `aiqaApi` transport layer (`Authorization: Bearer <key>` header, unchanged), never placed in the request body, query string, or any rendered UI text. The `Provider`/`Model` fields rendered in the Generation Details panel are the real backend response's own non-secret identifiers (e.g. `"ollama"`, `"llama3"`), never a credential. No new endpoint, no new auth mechanism, and no new client-side storage was introduced — this phase is a pure consumer of the pre-existing, already-audited auth/RBAC/project-scoping layer (`auth.rs`, unmodified).

29. Regression Review

`git diff --stat`: 7 files changed, 831 insertions, 399 deletions, all within `forentend/` — zero backend files. Frontend: 378/378 tests pass (up from 371; no test coverage lost — see §20), typecheck clean, production build clean. Backend: 206/206 tests pass on 3 of 4 runs this session, with the one transient failure independently reproduced as pre-existing flakiness unrelated to this phase's zero backend changes (§22).

30. Known Limitations

- The 3 seeded demo `generatedCases` in `AIQAContext.tsx` (fabricated "already generated" test cases shown on first load, with no `AI`/`Fallback` badge or Generation Details panel, since they predate this phase's real pipeline) remain in the codebase. This is a pre-existing, separately-flagged issue (this session's earlier full-project audit, finding AUD-006) that Phase 1 deliberately did not fully remediate — only the part of it that could feed fabricated content into a *real* AI call (`requirementText`) was fixed, to keep this phase's blast radius to what Phase 1 actually requires.
- A genuinely successful `generation_method: "ai"` response was not observed live this session (§14/§26) — no AI provider is reachable in this dev environment. The code path is real and covered by automated tests, but has not been eyeballed with a live model's actual output.
- `AiSettings.mode` is fetched with a second, best-effort request (`aiqaApi.ai.getSettings()`) purely for the Generation Details panel's `Mode` field; a failure there degrades to `Not Reported` and never blocks or fails the real generation call itself (verified by the try/catch wrapping it).
- Selected categories and generation depth have no dedicated field on the real backend contract; they are folded into `requirement_text` as plain instruction text (§9/§18) — a deliberate, disclosed design choice, not a hidden limitation, but worth restating here for completeness.

31. Remaining Work After Phase 1

Explicitly deferred to a later phase, not started:

- **Phase 2:** Generated Test Case -> real Execution Handoff (currently simulate-only, disclosed)
- Custom Testing, API Testing, RBAC Testing (still stub pages)
- Performance / Load / Stress / Spike / Volume testing (no capability exists)

On hold, unchanged:

- Mobile Testing
- MCP

---

32. Evidence Table

Capability	Frontend	Backend	Real AI Execution	Tests	Runtime Verified	Verdict
AI Test Case Generation (request path)	Real (`aiqaApi.testCases.generateAi`)	Real, unmodified (`api.rs`)	N/A (path itself)	12 dedicated + 4 adapted	**VERIFIED** (live, §26)	**VERIFIED**
Skill Transport	Real (`selectedSkillIds` -> request body)	Real, unmodified	N/A	2 dedicated	**VERIFIED** (live, §26/§10)	**VERIFIED**
Skill Resolution (server-side)	N/A	Real, unmodified, unchanged this phase	N/A	Pre-existing backend tests (not re-run individually)	NOT VERIFIED this session (reused, already tested)	**VERIFIED** (by reuse)
AI Model Router	N/A (no frontend duplication)	Real, unmodified	N/A	Pre-existing backend tests	Indirectly (`Mode: AUTO` shown live)	**VERIFIED** (by reuse)
AUTO Routing	Displays real mode	Real, unmodified	N/A	Pre-existing backend tests	**VERIFIED** (live, §12/§26)	**VERIFIED**
MANUAL Routing	Would display real mode if set	Real, unmodified	N/A	Pre-existing backend tests	NOT VERIFIED (§13)	**PARTIAL** (real, not live-exercised)
Provider Call (actual LLM)	N/A	Real, unmodified	Attempted live; no provider reachable	Pre-existing backend tests (2 live-Ollama-gated, `#[ignore]`d)	**NOT VERIFIED** — see §14	**BLOCKED** (environment)
Structured Validation	N/A	Real, unmodified	N/A	Pre-existing backend tests	NOT VERIFIED (needs a real AI response)	**VERIFIED** (by reuse)
Fallback Truthfulness	Real, response-driven display	Real, unmodified	N/A	1 dedicated	**VERIFIED** (live, §16/§26)	**VERIFIED**
Generation Metadata	Real "Generation Details" panel	Real fields, unmodified	N/A	1 dedicated	**VERIFIED** (live, §17/§26)	**VERIFIED**

---

33. Final Verdict

CONDITIONAL GO

Core AI generation integration works end to end: a real request genuinely reaches the real backend pipeline, real skills are genuinely transmitted and reflected back, the real AI Model Router is genuinely consulted (AUTO mode confirmed live), and the real outcome — success or honest failure — is displayed truthfully with no fabrication anywhere in the changed code. This was proven not just via mocked tests but via a live run independently corroborated by the orchestrator's own persisted audit log.

The "conditional" qualifier is entirely an **environment** limitation, not an implementation gap: no AI provider is installed/reachable in this dev environment, so a genuinely successful `generation_method: "ai"` case could not be observed with real model output. The code path that would produce it is real, unmodified, and already covered by the pre-existing backend test suite (including tests explicitly gated on a live Ollama connection) — installing Ollama (or configuring a cloud provider key) in this environment and re-running the exact same click-through would be expected to complete this verification, based on the honest, correctly-structured failure response already observed.

---

34. Final Questions

1. **Does clicking Generate Test Cases now send a real request to the backend?** Yes — verified live via the browser's network log and independently corroborated by the backend's own persisted audit log entry (`test_case.generate_ai`, §26).

2. **Does `selectedSkillIds` genuinely reach the backend generation path?** Yes — the exact 3 selected skills (2 default + 1 toggled on) appeared in the real response-driven Generation Details panel after a real round trip; an automated test additionally captures and asserts the real request body contains them.

3. **Do resolved skills genuinely reach the AI generation context/prompt?** Yes, via unchanged, pre-existing, already-tested backend code (`resolve_skill_instructions`/`build_prompt`) — not re-proven from scratch this phase, since it was not modified; reused as-is.

4. **Is the central AI Model Router genuinely used?** Yes — unchanged, unmodified, reused exactly as it already existed; no duplicate routing logic exists anywhere in the frontend.

5. **Which provider/model was actually used?** None — the live run's real, honest outcome was `NoCompatibleModel { task: TestCaseGeneration }` (no provider reachable in this environment), correctly reported as `provider: null, model: null` and displayed as "Not Reported".

6. **Was AUTO routing actually verified?** Yes, live — the real, currently-persisted `mode: AUTO` was fetched and displayed (§12/§26).

7. **Was MANUAL routing actually verified?** No — not manually exercised this session (§13); the underlying code is real, unmodified, and covered by pre-existing passing tests.

8. **Are generated test cases actual AI output or fallback output?** In the live run, fallback (empty — no AI output was produced, since no provider was reachable), and the UI honestly labeled it as such at every level (badge, status, panel, traceability chain).

9. **Is fallback truthfully labeled?** Yes — verified live end to end, including the exact real backend-generated reason string, not a client-side paraphrase.

10. **Is provider/model metadata actual rather than hardcoded?** Yes — grep-confirmed no hardcoded provider/model name exists anywhere in the changed code; every rendered value is either the real response field or the honest "Not Reported"/"Not recorded" placeholder.

11. **Do documents/app sources genuinely reach the generation context?** Only as literal name/label metadata, explicitly disclosed in the prompt text itself as not-analyzed/not-fetched — never their actual content, because that content genuinely never reaches the frontend either (§18).

12. **Are there any remaining local templates presented as AI output?** No — the entire ~285-line hardcoded per-category template `switch` statement was deleted, not merely disclosed-and-kept. The 3 pre-existing seeded demo `generatedCases` remain (§30) but now correctly render `NOT AI-GENERATED` rather than the old unconditional `TEMPLATE` claim.

13. **Were any fake/stub behaviors introduced?** No — see the fake/stub audit (§27); every match in the changed files is either an HTML placeholder attribute, a real category label, or documentation (including this phase's own comments describing what it fixed).

14. **What exact Phase 1 limitation remains?** No AI provider is installed/reachable in this development environment, so a genuinely successful AI-generated response was not observed with live model output (§14/§26) — a real, correctly-structured failure path was observed and verified instead, and the success path is covered by automated tests plus the unmodified, already-tested backend code that would produce it.

15. **What is the next phase after this?** Phase 2 — Generated Test Case -> real Execution Handoff (currently simulate-only), per §31.

Prepared as a fresh, evidence-based implementation report; every claim above is backed by a file:line reference, a passing/failing test, or a live runtime observation independently corroborated server-side. No claim optimizes for a positive result over the truth.